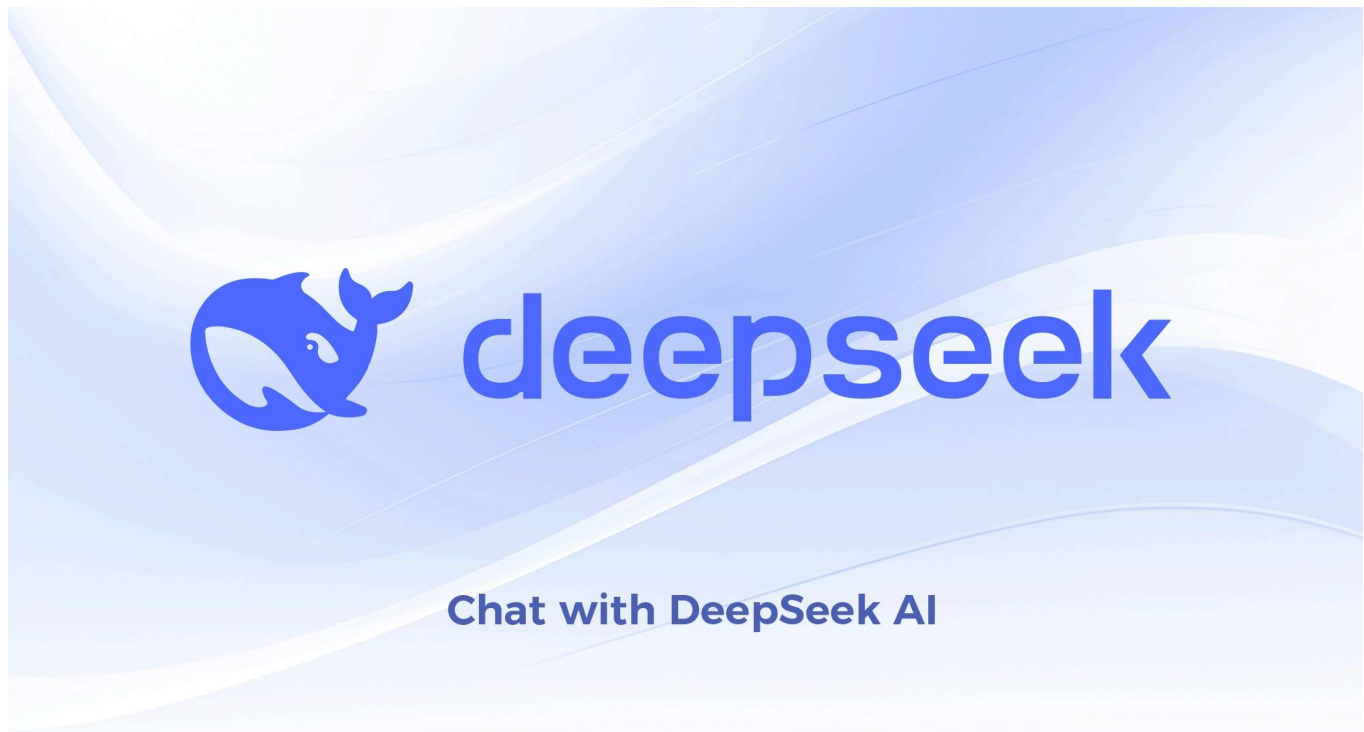


Python Inheritance Interview Guide - DeepSeek



INHERITANCE CONCEPT AND CODE FOR INTERVIEW IN PYTHON

I'll provide a comprehensive guide to inheritance in Python, covering concepts and practical code examples for interviews.

INHERITANCE CONCEPTS FOR INTERVIEW

What is Inheritance?

Inheritance is an OOP concept where a class (child/derived) acquires properties and behaviors from another class (parent/base). It promotes code reusability and establishes relationships between classes.

Key Concepts:

1. **IS-A Relationship:** Child IS-A type of parent
2. **Method Resolution Order (MRO):** Order in which methods are searched
3. **super():** Access parent class methods
4. **Method Overriding:** Child redefines parent method

5. **Types:** Single, Multiple, Multilevel, Hierarchical, Hybrid

PRACTICAL CODE EXAMPLES

1. Basic Single Inheritance

python

```
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def make_sound(self):
        return "Some generic animal sound"

    def describe(self):
        return f"{self.name} is a {self.species}"

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, species="Canine")
        self.breed = breed

    def make_sound(self):
        return "Woof! Woof!"

    def fetch(self):
        return f"{self.name} is fetching the ball"

dog = Dog("Buddy", "Golden Retriever")
print(dog.describe())
print(dog.make_sound())
print(dog.fetch())
```

2. Multiple Inheritance with MRO

python

```
class Father:
    def __init__(self):
        self.father_name = "John"

    def skills(self):
        return "Programming, Mathematics"

    def height(self):
        return "6 feet"

class Mother:
    def __init__(self):
        self.mother_name = "Mary"

    def skills(self):
        return "Cooking, Painting"

    def height(self):
        return "5 feet 4 inches"

class Child(Father, Mother):
    def __init__(self, name):
        Father.__init__(self)
        Mother.__init__(self)
        self.name = name

    def skills(self):

        father_skills = Father.skills(self)
        mother_skills = Mother.skills(self)
        return f"Father: {father_skills}, Mother: {mother_skills}"

child = Child("Alex")
print(child.name)
print(child.skills())
print(child.height())
print(Child.__mro__)
```

3. Multilevel Inheritance

python

```
class LivingBeing:
    def breathe(self):
        return "Breathing oxygen"

class Animal(LivingBeing):
    def move(self):
        return "Moving around"

class Bird(Animal):
    def fly(self):
        return "Flying in the sky"

class Sparrow(Bird):
    def chirp(self):
        return "Chirp chirp!"

sparrow = Sparrow()
print(sparrow.breathe())
print(sparrow.move())
print(sparrow.fly())
print(sparrow.chirp())
```

4. super() Deep Dive

python


```
class Shape:
    def __init__(self, color):
        self.color = color
        print(f"Shape initialized with color: {color}")

    def area(self):
        return 0

class Rectangle(Shape):
    def __init__(self, color, width, height):
        super().__init__(color)
        self.width = width
        self.height = height
        print(f"Rectangle created: {width}x{height}")

    def area(self):
        return self.width * self.height

class Square(Rectangle):
    def __init__(self, color, side):
        super().__init__(color, side, side)
        print(f"Square created with side: {side}")

square = Square("Red", 5)
print(f"Area: {square.area()}")
print(f"Color: {square.color}")
```

5. Abstract Base Classes

python

```
from abc import ABC, abstractmethod

class PaymentProcessor(ABC):
    def __init__(self, amount):
        self.amount = amount

    @abstractmethod
    def process_payment(self):
        """Every payment processor must implement this method"""
        pass

    @abstractmethod
    def refund(self):
        """Every payment processor must implement this method"""
        pass

    def validate_amount(self):
        return self.amount > 0

class CreditCardPayment(PaymentProcessor):
    def __init__(self, amount, card_number):
        super().__init__(amount)
        self.card_number = card_number

    def process_payment(self):
        return f"Processing ${self.amount} from card {self.card_number[-4:]}"

    def refund(self):
        return f"Refunding ${self.amount} to card {self.card_number[-4:]}"

class PayPalPayment(PaymentProcessor):
    def __init__(self, amount, email):
        super().__init__(amount)
        self.email = email

    def process_payment(self):
        return f"Processing ${self.amount} via PayPal from {self.email}"

    def refund(self):
        return f"Refunding ${self.amount} to {self.email}"

credit = CreditCardPayment(150, "1234-5678-9012-3456")
paypal = PayPalPayment(75, "user@example.com")
```

```
print(credit.process_payment())  
print(payload.process_payment())
```

6. Real-World Example: Employee Management System

python

```
from datetime import datetime

class Employee:
    raise_amount = 1.04

    def __init__(self, name, salary, join_date):
        self.name = name
        self.salary = salary
        self.join_date = join_date

    def apply_raise(self):
        self.salary = int(self.salary * self.raise_amount)
        return self.salary

    def get_tenure(self):
        tenure = datetime.now() - self.join_date
        return tenure.days // 365

    def __str__(self):
        return f"{self.name} - {self.designation}"

class Manager(Employee):
    raise_amount = 1.10

    def __init__(self, name, salary, join_date, team_size):
        super().__init__(name, salary, join_date)
        self.team_size = team_size
        self.designation = "Manager"

    def manage_team(self):
        return f"Managing {self.team_size} employees"

class Developer(Employee):
    raise_amount = 1.08

    def __init__(self, name, salary, join_date, programming_languages):
        super().__init__(name, salary, join_date)
        self.programming_languages = programming_languages
        self.designation = "Developer"

    def add_language(self, language):
        if language not in self.programming_languages:
            self.programming_languages.append(language)

    def get_languages(self):
        return ", ".join(self.programming_languages)

class SeniorDeveloper(Developer):
    raise_amount = 1.12
```

```
def __init__(self, name, salary, join_date, programming_languages, mentees):
    super().__init__(name, salary, join_date, programming_languages)
    self.mentees = mentees
    self.designation = "Senior Developer"

def mentor(self):
    return f"Mentoring {len(self.mentees)} junior developers"

manager = Manager("John Doe", 80000, datetime(2020, 1, 15), 10)
dev = Developer("Jane Smith", 65000, datetime(2021, 6, 1), ["Python", "JavaScript"])
senior_dev = SeniorDeveloper("Bob Wilson", 90000, datetime(2019, 3, 10),
                             ["Python", "Java", "Go"], ["Alice", "Charlie"])

print(manager.apply_raise())
print(dev.apply_raise())
print(senior_dev.apply_raise())

print(isinstance(senior_dev, Employee))
print(isinstance(senior_dev, Developer))
print(isinstance(manager, Developer))
print(issubclass(SeniorDeveloper, Employee))
```

7. Diamond Problem and Resolution

python

```
class A:
    def method(self):
        return "A method"

    def common(self):
        return "Common from A"

class B(A):
    def method(self):
        return "B method"

    def common(self):
        return "Common from B"

class C(A):
    def method(self):
        return "C method"

    def common(self):
        return "Common from C"

class D(B, C):
    def common(self):

        b_result = B.common(self)
        c_result = C.common(self)
        return f"{b_result} and {c_result}"

d = D()
print(D.__mro__)
print(d.method())
print(d.common())
```

Interview Questions & Quick Answers

Common Interview Questions:

1. Q: What's the difference between `super().__init__()` and `Parent.__init__(self)`?

python

```
class Child(Parent):
    def __init__(self):
        super().__init__()
```

2. Q: How to make methods private-like?

python

```
class MyClass:
    def _protected_method(self):
        pass

    def __private_method(self):
        pass
```

3. Q: What is `isinstance()` vs `issubclass()`?

python

```
isinstance(obj, Class)
issubclass(Child, Parent)
```

4. Q: Can you inherit from built-in types?

python

```
class MyList(list):
    def __init__(self, *args):
        super().__init__(args)

    def prepend(self, item):
        self.insert(0, item)

ml = MyList(1, 2, 3)
ml.append(4)
ml.prepend(0)
print(ml)
```

Key Interview Points:

1. **`super()`** ensures proper MRO in multiple inheritance
2. **Composition over Inheritance** - consider using composition when relationship isn't clear "IS-A"
3. **Diamond Problem** - Python uses C3 Linearization to resolve
4. **Abstract classes** enforce implementation in child classes

5. **Method resolution order** can be viewed with `ClassName.__mro__` or `ClassName.mro()`

This covers the essential inheritance concepts you'll need for Python interviews. Practice implementing these patterns and be prepared to discuss the pros and cons of inheritance versus composition!

INHERITANCE ITS and very easy word def and CONCEPT AND CODE FOR INTERVIEW IN PYTHON

INHERITANCE IN PYTHON - Simple Explanation with Interview Code

What is Inheritance? (Super Simple Definition)

Inheritance = Child class gets everything from Parent class, like a child inherits property from parents.

Simple Analogy:

- Parent has a house and car
- Child automatically gets that house and car (inherits)
- Child can also buy new things (add new features)
- Child can modify the car (override)

Core Concepts in Simple Words

1. Why Inheritance?

- **Code Reuse:** Don't write same code again
- **Relationship:** Shows "IS-A" relation (Car IS-A Vehicle)
- **Organization:** Group similar things together

2. Basic Terms

- **Parent Class** (Base/Super class): The class giving properties
- **Child Class** (Derived/Sub class): The class taking properties

- **extends/inherit**s: Child gets parent's features

1. BASIC INHERITANCE - The Foundation

python

```
class Vehicle:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color

    def start(self):
        return "Vehicle started"

    def stop(self):
        return "Vehicle stopped"

    def info(self):
        return f"{self.color} {self.brand}"

class Car(Vehicle):
    def __init__(self, brand, color, doors):
        super().__init__(brand, color)
        self.doors = doors

    def honk(self):
        return "Beep Beep!"

    def info(self):
        basic_info = super().info()
        return f"{basic_info} with {self.doors} doors"

my_car = Car("Toyota", "Red", 4)

print(my_car.start())
print(my_car.stop())
print(my_car.honk())
print(my_car.info())
```

What happened?

- Car got **start()**, **stop()** automatically from Vehicle
- Car added its own **honk()** method

- Car modified `info()` to show doors

2. SUPER() EXPLAINED SIMPLY

python

```
class Animal:
    def __init__(self, name):
        self.name = name
        print(f"Animal {name} created")

    def eat(self):
        return f"{self.name} is eating"

class Dog(Animal):
    def __init__(self, name, breed):

        super().__init__(name)
        self.breed = breed
        print(f"Dog breed: {breed}")

    def eat(self):

        parent_result = super().eat()
        return f"{parent_result} dog food"

dog = Dog("Buddy", "Labrador")

print(dog.eat())
```

super() in one line: "Call my parent's version of this method"

3. METHOD OVERRIDING - Change Parent's Behavior

python

```
class Phone:
    def call(self):
        return "Making a phone call"

    def message(self):
        return "Sending SMS"

class SmartPhone(Phone):

    def call(self):
        return "Video calling with HD quality"

    def message(self):
        old_way = super().message()
        return f"{old_way} and WhatsApp"

basic_phone = Phone()
smart_phone = SmartPhone()

print(basic_phone.call())
print(smart_phone.call())
print(basic_phone.message())
```

4. TYPES OF INHERITANCE - Visual Examples

python

```
class Father:
    def skill(self):
        return "Cooking"
```

```
class Son(Father):
    pass
```

```
son = Son()
print(son.skill())
```

```
class Grandfather:
    def tradition(self):
        return "Family values"
```

```
class Father(Grandfather):
    def profession(self):
        return "Doctor"
```

```
class Son(Father):
    def hobby(self):
        return "Guitar"
```

```
son = Son()
print(son.tradition())
print(son.profession())
print(son.hobby())
```

```
class Animal:
    def breathe(self):
        return "Breathing"
```

```
class Dog(Animal):
    def sound(self):
        return "Bark"
```

```
class Cat(Animal):
    def sound(self):
        return "Meow"
```

```
dog = Dog()
cat = Cat()
print(dog.breathe())
```

```
print(cat.breathe())
print(dog.sound())
print(cat.sound())

class Father:
    def height(self):
        return "Tall"

class Mother:
    def intelligence(self):
        return "Smart"

class Child(Father, Mother):
    def creativity(self):
        return "Creative"

child = Child()
print(child.height())
print(child.intelligence())
print(child.creativity())
```

5. PRACTICAL EXAMPLES FOR INTERVIEW

Example 1: Bank Account System

python

```
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return f"Deposited ${amount}. New balance: ${self.balance}"

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            return f"Withdrew ${amount}. New balance: ${self.balance}"
        return "Insufficient funds!"

    def check_balance(self):
        return f"Balance: ${self.balance}"

class SavingsAccount(BankAccount):
    def __init__(self, account_holder, balance, interest_rate):
        super().__init__(account_holder, balance)
        self.interest_rate = interest_rate

    def add_interest(self):
        interest = self.balance * (self.interest_rate / 100)
        self.balance += interest
        return f"Added interest: ${interest}. New balance: ${self.balance}"

class CurrentAccount(BankAccount):
    def __init__(self, account_holder, balance, overdraft_limit):
        super().__init__(account_holder, balance)
        self.overdraft_limit = overdraft_limit

    def withdraw(self, amount):
        if amount <= self.balance + self.overdraft_limit:
            self.balance -= amount
            return f"Withdrew ${amount}. Balance: ${self.balance}"
        return "Overdraft limit exceeded!"

savings = SavingsAccount("Alice", 1000, 5)
current = CurrentAccount("Bob", 2000, 500)

print(savings.add_interest())
print(current.withdraw(2200))
```

Example 2: Employee Payroll System

python

```
class Employee:
    def __init__(self, name, id, base_salary):
        self.name = name
        self.id = id
        self.base_salary = base_salary

    def calculate_salary(self):
        return self.base_salary

    def get_role(self):
        return "Employee"

class Manager(Employee):
    def __init__(self, name, id, base_salary, team_size):
        super().__init__(name, id, base_salary)
        self.team_size = team_size

    def calculate_salary(self):

        bonus = self.team_size * 1000
        return self.base_salary + bonus

    def get_role(self):
        return "Manager"

class SalesPerson(Employee):
    def __init__(self, name, id, base_salary, commission_rate, sales):
        super().__init__(name, id, base_salary)
        self.commission_rate = commission_rate
        self.sales = sales

    def calculate_salary(self):

        commission = self.sales * (self.commission_rate / 100)
        return self.base_salary + commission

    def get_role(self):
        return "Sales Person"

employees = [
    Manager("John", 101, 50000, 5),
    SalesPerson("Jane", 102, 30000, 10, 50000),
    Employee("Bob", 103, 40000)
]

for emp in employees:
    print(f'{emp.get_role()} {emp.name}: ${emp.calculate_salary()}')
```


6. COMMON INTERVIEW QUESTIONS & ANSWERS

Q1: What is the difference between **super()** and direct parent call?

python

```
class Parent:
    def __init__(self):
        print("Parent init")

class Child1(Parent):
    def __init__(self):
        Parent.__init__(self)
        print("Child1 init")

class Child2(Parent):
    def __init__(self):
        super().__init__()
        print("Child2 init")

class A:
    def __init__(self):
        print("A init")

class B(A):
    def __init__(self):
        super().__init__()
        print("B init")

class C(A):
    def __init__(self):
        super().__init__()
        print("C init")

class D(B, C):
    def __init__(self):
        super().__init__()
        print("D init")

D()
```

Q2: What is Method Resolution Order (MRO)?

python

```
class A:
    def greet(self):
        return "Hello from A"

class B(A):
    def greet(self):
        return "Hello from B"

class C(A):
    def greet(self):
        return "Hello from C"

class D(B, C):
    pass

d = D()
print(d.greet())
print(D.__mro__)
```

Q3: How to prevent inheritance?

python

```
from typing import final

@final
class CannotInherit:
    pass

class PreventInheritance:
    def __init_subclass__(cls, **kwargs):
        raise TypeError("This class cannot be inherited")
```

Q4: isinstance() and issubclass()

python

```
class Animal:
    pass

class Dog(Animal):
    pass

class Cat(Animal):
    pass

dog = Dog()

print(isinstance(dog, Dog))
print(isinstance(dog, Animal))
print(isinstance(dog, Cat))

print(issubclass(Dog, Animal))
print(issubclass(Dog, Cat))
print(issubclass(Animal, Dog))
```

7. DO's AND DON'Ts

DO:

python

```
class Bird(Animal):
    pass

class Child(Parent):
    def __init__(self):
        super().__init__()

class A:
    pass
class B(A):
    pass
class C(B):
    pass
```

DON'T:

python

```

class Car:
    def __init__(self):
        self.engine = Engine()

class A:
    pass
class B(A):
    pass
class C(B):
    pass
class D(C):
    pass

class Child(Parent):
    def method1(self):
        pass
    def method2(self):
        pass

```

QUICK CHEAT SHEET FOR INTERVIEW

Remember This Mantra:

"Child inherits everything from Parent, can use as-is, modify it, or add new things!"

This simple explanation covers everything you need for interviews! The key is understanding the "IS-A" relationship and using `super()` correctly.

This response is AI-generated, for reference only.

polymorphism ITS and very easy word def and CONCEPT AND CODE FOR INTERVIEW IN PYTHON

POLYMORPHISM IN PYTHON - Simple Explanation with Interview Code

What is Polymorphism? (Super Simple Definition)

Poly = Many, Morph = Forms → One thing taking many forms

Simple Analogy:

- **You** act differently in different places:
 - At home → You're a family member
 - At office → You're an employee
 - At party → You're a friend
 - **Same person, different behaviors!**

In Programming:

- Same method name, but different classes do different things
- Like a remote control's "Play" button:
 - On TV → Plays video
 - On Music Player → Plays song
 - On Game Console → Plays game
 - **Same button "Play", different actions!**

Core Concepts in Simple Words

1. Why Polymorphism?

- **Flexibility:** Use same interface for different types
- **Simplicity:** One method name, multiple behaviors
- **Extensibility:** Add new classes without changing existing code

2. Two Main Rules:

- **Same Method Name:** Different classes have same method name
- **Different Behavior:** Each class implements it differently

1. BASIC POLYMORPHISM - Same Method, Different Actions

python

```
class Dog:
    def speak(self):
        return "Woof! Woof!"

    def move(self):
        return "Runs on 4 legs"

class Cat:
    def speak(self):
        return "Meow! Meow!"

    def move(self):
        return "Walks silently"

class Bird:
    def speak(self):
        return "Chirp! Chirp!"

    def move(self):
        return "Flies in sky"

animals = [Dog(), Cat(), Bird()]

for animal in animals:
    print(f"Speak: {animal.speak()}")
    print(f"Move: {animal.move()}")
    print("---")
```

What happened?

- All animals have `speak()` and `move()` methods
- But each does it differently

- We can call same methods without knowing what animal it is

2. POLYMORPHISM WITH INHERITANCE - Parent-Child Style

python

```
class Shape:
    def area(self):
        return 0

    def perimeter(self):
        return 0

    def describe(self):
        return f"I am a shape with area: {self.area()}"
```

```
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

    def perimeter(self):
        return 2 * 3.14 * self.radius

    def describe(self):
        return f"Circle with radius {self.radius}"
```

```
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def describe(self):
        return f"Rectangle {self.width}x{self.height}"
```

```
class Triangle(Shape):
    def __init__(self, base, height, side1, side2):
        self.base = base
        self.height = height
        self.side1 = side1
        self.side2 = side2

    def area(self):
        return 0.5 * self.base * self.height

    def perimeter(self):
```



```
        return self.base + self.side1 + self.side2

    def describe(self):
        return f"Triangle with base {self.base}"

shapes = [
    Circle(5),
    Rectangle(4, 6),
    Triangle(3, 4, 5, 5)
]

for shape in shapes:
    print(shape.describe())
    print(f"Area: {shape.area()}")
    print(f"Perimeter: {shape.perimeter()}")
    print("----")
```

3. REAL-WORLD EXAMPLE: Payment System

python

```
class Payment:
    def pay(self, amount):
        pass

    def refund(self, amount):
        pass

    def get_payment_info(self):
        pass

class CreditCardPayment(Payment):
    def __init__(self, card_number, cvv):
        self.card_number = card_number
        self.cvv = cvv

    def pay(self, amount):
        return f"Paid ${amount} using Credit Card ending with {self.card_number[-4:]}"

    def refund(self, amount):
        return f"Refunded ${amount} to Credit Card ending with {self.card_number[-4:]}"

    def get_payment_info(self):
        return f"Credit Card: ****{self.card_number[-4:]}"

class PayPalPayment(Payment):
    def __init__(self, email):
        self.email = email

    def pay(self, amount):
        return f"Paid ${amount} using PayPal account {self.email}"

    def refund(self, amount):
        return f"Refunded ${amount} to PayPal account {self.email}"

    def get_payment_info(self):
        return f"PayPal: {self.email}"

class UpiPayment(Payment):
    def __init__(self, upi_id):
        self.upi_id = upi_id

    def pay(self, amount):
        return f"Paid ${amount} using UPI {self.upi_id}"

    def refund(self, amount):
        return f"Refunded ${amount} to UPI {self.upi_id}"

    def get_payment_info(self):
        return f"UPI: {self.upi_id}"
```

```

class CryptoPayment(Payment):
    def __init__(self, wallet_address):
        self.wallet_address = wallet_address

    def pay(self, amount):
        return f"Paid ${amount} using Crypto wallet {self.wallet_address[:10]}..."

    def refund(self, amount):
        return f"Refunded ${amount} to Crypto wallet {self.wallet_address[:10]}..."

    def get_payment_info(self):
        return f"Crypto: {self.wallet_address[:10]}..."

def process_payment(payment_method, amount):
    print(payment_method.get_payment_info())
    print(payment_method.pay(amount))
    print("---")

payments = [
    CreditCardPayment("1234567890123456", "123"),
    PayPalPayment("user@example.com"),
    UpiPayment("user@bank"),
    CryptoPayment("0x1234567890abcdef")
]

for payment in payments:
    process_payment(payment, 100)

```

4. DUCK TYPING - Python's Special Polymorphism

"If it walks like a duck and quacks like a duck, it's a duck!"

Python doesn't care about type, only cares if method exists!

python

```
class Duck:
    def sound(self):
        return "Quack Quack!"

    def swim(self):
        return "Duck swimming"

class Person:
    def sound(self):
        return "I can quack like a duck!"

    def swim(self):
        return "Person swimming"

    def talk(self):
        return "Hello!"

class Dog:
    def sound(self):
        return "Woof Woof!"

    def swim(self):
        return "Dog paddling"

def make_it_quack_and_swim(thing):
    print(thing.sound())
    print(thing.swim())
    print("---")

make_it_quack_and_swim(Duck())
make_it_quack_and_swim(Person())
make_it_quack_and_swim(Dog())
```

Key Point: Python doesn't care WHAT you are, only cares WHAT you can do!

5. METHOD OVERLOADING (Not in Python but Important)

python

```
class Calculator:
```

```
    def add(self, a, b, c=0):  
        return a + b + c
```

```
    def multiply(self, *args):  
        result = 1  
        for num in args:  
            result *= num  
        return result
```

```
calc = Calculator()
```

```
print(calc.add(10, 20))  
print(calc.add(10, 20, 30))
```

```
print(calc.multiply(2, 3))  
print(calc.multiply(2, 3, 4))  
print(calc.multiply(2, 3, 4, 5))
```

```
from functools import singledispatchmethod
```

```
class AdvancedCalculator:
```

```
    @singledispatchmethod  
    def process(self, arg):  
        return f"Default: {arg}"
```

```
    @process.register  
    def _(self, arg: int):  
        return f"Integer: {arg * 2}"
```

```
    @process.register  
    def _(self, arg: str):  
        return f"String: {arg.upper()}"
```

```
    @process.register  
    def _(self, arg: list):  
        return f"List: {len(arg)} items"
```

```
calc2 = AdvancedCalculator()  
print(calc2.process(10))  
print(calc2.process("hello"))  
print(calc2.process([1, 2, 3]))
```

6. OPERATOR OVERLOADING - Polymorphism with Operators

python

```
print(10 + 20)
print("Hello " + "World")
print([1, 2] + [3, 4])
```

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __mul__(self, scalar):
        return Vector(self.x * scalar, self.y * scalar)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

    def __len__(self):
        return 2

class Money:
    def __init__(self, amount, currency):
        self.amount = amount
        self.currency = currency

    def __add__(self, other):
        if self.currency == other.currency:
            return Money(self.amount + other.amount, self.currency)
        else:
            raise ValueError("Cannot add different currencies")

    def __str__(self):
        return f"{self.currency} {self.amount}"

v1 = Vector(2, 3)
v2 = Vector(4, 5)
v3 = v1 + v2
```

```
v4 = v1 * 3

print(v3)
print(v4)
print(v1 == v2)
print(len(v1))

m1 = Money(100, "USD")
m2 = Money(200, "USD")
m3 = m1 + m2
print(m3)
```

7. COMMON OPERATOR OVERLOADING METHODS

python


```
class CustomClass:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return CustomClass(self.value + other.value)

    def __sub__(self, other):
        return CustomClass(self.value - other.value)

    def __mul__(self, other):
        return CustomClass(self.value * other.value)

    def __truediv__(self, other):
        return CustomClass(self.value / other.value)

    def __eq__(self, other):
        return self.value == other.value

    def __lt__(self, other):
        return self.value < other.value

    def __gt__(self, other):
        return self.value > other.value

    def __str__(self):
        return f"CustomClass({self.value})"

    def __repr__(self):
        return f"CustomClass({self.value})"

    def __len__(self):
        return abs(self.value)

    def __getitem__(self, index):
        return self.value

    def __call__(self):
        return f"Called with {self.value}"

c1 = CustomClass(10)
c2 = CustomClass(20)

print(c1 + c2)
```

```
print(c1 * c2)
print(c1 < c2)
print(len(c1))
print(c1())
```

8. REAL INTERVIEW EXAMPLE: File Processing System

python

```
class FileProcessor:
    def process(self, file_path):
        """Process file - to be implemented by subclasses"""
        raise NotImplementedError("Subclass must implement process method")

    def get_file_type(self):
        return "Generic"

class CSVProcessor(FileProcessor):
    def process(self, file_path):
        return f"Processing CSV file: {file_path} - Parsing comma-separated values"

    def get_file_type(self):
        return "CSV"

class JSONProcessor(FileProcessor):
    def process(self, file_path):
        return f"Processing JSON file: {file_path} - Parsing JSON structure"

    def get_file_type(self):
        return "JSON"

class XMLProcessor(FileProcessor):
    def process(self, file_path):
        return f"Processing XML file: {file_path} - Parsing XML tags"

    def get_file_type(self):
        return "XML"

class ImageProcessor(FileProcessor):
    def process(self, file_path):
        return f"Processing Image file: {file_path} - Resizing and optimizing"

    def get_file_type(self):
        return "Image"

def get_processor(file_path):
    extension = file_path.split('.')[-1].lower()

    processors = {
        'csv': CSVProcessor(),
        'json': JSONProcessor(),
        'xml': XMLProcessor(),
        'jpg': ImageProcessor(),
        'png': ImageProcessor(),
        'gif': ImageProcessor()
    }

    return processors.get(extension, FileProcessor())
```

```
files = [  
    "data.csv",  
    "config.json",  
    "document.xml",  
    "photo.jpg",  
    "unknown.xyz"  
]  
  
for file in files:  
    processor = get_processor(file)  
    print(f"Type: {processor.get_file_type()}")  
    print(processor.process(file))  
    print("---")
```

9. POLYMORPHISM WITH FUNCTIONS

python

```
def calculate_total_area(shapes):
    """Works with ANY object that has area() method"""
    total = 0
    for shape in shapes:
        total += shape.area()
    return total

def make_sound_twice(animal):
    """Works with ANY object that has sound() method"""
    return f"{animal.sound()} {animal.sound()}"

class Circle:
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2

class Square:
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2

class CustomShape:
    def __init__(self):
        pass
    def area(self):
        return 100

shapes = [Circle(5), Square(4), CustomShape()]
print(f"Total area: {calculate_total_area(shapes)}")

class Dog:
    def sound(self):
        return "Woof"

class Cat:
    def sound(self):
        return "Meow"

class Car:
    def sound(self):
        return "Beep Beep"

print(make_sound_twice(Dog()))
```

```
print(make_sound_twice(Cat()))  
print(make_sound_twice(Car()))
```

10. COMMON INTERVIEW QUESTIONS & ANSWERS

Q1: What is the difference between Inheritance and Polymorphism?

python

```
class Animal:  
    def eat(self):  
        return "Eating"  
  
class Dog(Animal):  
    pass  
  
class Dog:  
    def speak(self):  
        return "Woof"  
  
class Cat:  
    def speak(self):  
        return "Meow"
```

Q2: How is Duck Typing different from traditional polymorphism?

python

```
from abc import ABC, abstractmethod
```

```
class Bird(ABC):  
    @abstractmethod  
    def fly(self):  
        pass
```

```
class Sparrow(Bird):  
    def fly(self):  
        return "Sparrow flying"
```

```
class Sparrow:  
    def fly(self):  
        return "Sparrow flying"
```

```
class Airplane:  
    def fly(self):  
        return "Airplane flying"
```

```
class Superman:  
    def fly(self):  
        return "Superman flying"
```

```
def make_it_fly(thing):  
    return thing.fly()
```

```
print(make_it_fly(Sparrow()))  
print(make_it_fly(Airplane()))  
print(make_it_fly(Superman()))
```

Q3: What is method overriding vs method overloading?

python

```
class Parent:
    def greet(self):
        return "Hello from Parent"

class Child(Parent):
    def greet(self):
        return "Hello from Child"

class Example:
    def greet(self, name=None):
        if name is None:
            return "Hello!"
        else:
            return f"Hello {name}!"

obj = Example()
print(obj.greet())
print(obj.greet("John"))
```

Q4: Practical use of str vs repr

python

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Point at ({self.x}, {self.y})"

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

p = Point(3, 4)
print(str(p))
print(repr(p))

points = [Point(1, 2), Point(3, 4)]
print(points)
```


11. DO's AND DON'Ts

✓ DO:

python

```
class EmailSender:
    def send(self, message):
        pass

class SMTPEmail(EmailSender):
    def send(self, message):
        return f"Sending via SMTP: {message}"

class SESEmail(EmailSender):
    def send(self, message):
        return f"Sending via AWS SES: {message}"

class Database:
    def connect(self, host, port):
        pass

class MySQL(Database):
    def connect(self, host, port):
        return f"MySQL: {host}:{port}"

def process_data(data):
    return data.to_json()
```

✗ DON'T:

python

```
class Parent:
    def method(self, x, y):
        pass

class Child(Parent):
    def method(self, x):
        pass

def process(shape):
    if type(shape) == Circle:

        pass
    elif type(shape) == Square:

        pass

def process(shape):
    return shape.area()

class Bad:
    def __add__(self, other):
        return "This doesn't make sense!"
```

12. QUICK CHEAT SHEET FOR INTERVIEW

Remember This Mantra:

"One Interface, Many Implementations"

Same method call → Different behaviors for different objects

Key Interview Points:

1. Polymorphism = flexibility + code reuse
2. Duck typing = Python's unique polymorphism
3. Don't check types, check behavior
4. Use abstract classes to enforce interfaces
5. Operator overloading makes custom classes feel like built-ins

Polymorphism is about **what an object can DO**, not **what an object IS**!

Abstraction ITS and very easy word def and CONCEPT AND CODE FOR INTERVIEW IN PYTHON

ABSTRACTION IN PYTHON - Simple Explanation with Interview Code

What is Abstraction? (Super Simple Definition)

| Abstraction = Hide complexity, show only what's necessary

Simple Analogy:

- **TV Remote:** You press "Power" button → TV turns on
 - You DON'T need to know about circuits, electricity, signals
 - You only need to know WHICH button to press
 - **Complexity hidden, simple interface shown!**
- **Car Driving:** You turn steering wheel → Car turns
 - You DON'T need to know engine mechanics, gear systems
 - You only need to know HOW to drive
 - **Internal details hidden, essential features exposed!**

In Programming:

- Hide implementation details
- Show only important methods
- Like a restaurant menu: You see dishes (interface), not recipes (implementation)

Core Concepts in Simple Words

1. Why Abstraction?

- **Simplicity:** Users don't need to know complex internals
- **Security:** Hide sensitive implementation details
- **Flexibility:** Change internal code without affecting users

- **Focus:** Concentrate on WHAT object does, not HOW

2. Two Key Ideas:

- **What to show:** Essential features (interface)
- **What to hide:** Complex implementation details

1. BASIC ABSTRACTION - Hiding Details

python

```
class CoffeeMachine_Bad:
    def __init__(self):
        self.water_level = 0
        self.coffee_beans = 0
        self.temperature = 0

    def heat_water(self):
        self.temperature = 90
        print("Heating water to 90°C")

    def grind_beans(self):
        print("Grinding coffee beans")

    def brew(self):
        print("Brewing coffee")

    def pour(self):
        print("Pouring coffee")

    def clean(self):
        print("Cleaning machine")
```

```
machine = CoffeeMachine_Bad()
machine.heat_water()
machine.grind_beans()
machine.brew()
machine.pour()
machine.clean()
```

```
class CoffeeMachine:
    def __init__(self):
        self.__water_level = 0
        self.__coffee_beans = 0
        self.__temperature = 0

    def __heat_water(self):
        self.__temperature = 90
        print("Heating water...")

    def __grind_beans(self):
        print("Grinding beans...")

    def __brew(self):
        print("Brewing...")
```

```
def __pour(self):
    print("Pouring...")

def __clean(self):
    print("Auto-cleaning...")

def make_coffee(self):
    """User only needs to know THIS method!"""
    print("Making your coffee...")
    self.__heat_water()
    self.__grind_beans()
    self.__brew()
    self.__pour()
    self.__clean()
    print("☕ Coffee is ready! Enjoy!")
```

```
machine = CoffeeMachine()
machine.make_coffee()
```

What happened?

- User sees only `make_coffee()`
- All complex steps are hidden
- User doesn't need to know internal process

2. ABSTRACTION USING ABSTRACT CLASSES

python

```
from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def make_sound(self):
        """Every animal MUST implement this"""
        pass

    @abstractmethod
    def move(self):
        """Every animal MUST implement this"""
        pass

    def sleep(self):
        return f"{self.name} is sleeping"

    def eat(self):
        return f"{self.name} is eating"

class Dog(Animal):
    def make_sound(self):
        return f"{self.name} says: Woof Woof!"

    def move(self):
        return f"{self.name} runs on 4 legs"

class Bird(Animal):
    def make_sound(self):
        return f"{self.name} says: Chirp Chirp!"

    def move(self):
        return f"{self.name} flies in the sky"

class Fish(Animal):
    def make_sound(self):
        return f"{self.name} says: Blub Blub!"

    def move(self):
        return f"{self.name} swims in water"

animals = [
```

```
Dog("Buddy"),  
Bird("Tweety"),  
Fish("Nemo")  
]  
  
for animal in animals:  
    print(animal.make_sound())  
    print(animal.move())  
    print(animal.sleep())  
    print(animal.eat())  
    print("---")
```

3. REAL-WORLD EXAMPLE: Database Connection

python


```
from abc import ABC, abstractmethod

class Database(ABC):
    @abstractmethod
    def connect(self):
        """Connect to database"""
        pass

    @abstractmethod
    def disconnect(self):
        """Disconnect from database"""
        pass

    @abstractmethod
    def execute_query(self, query):
        """Execute a query"""
        pass

    @abstractmethod
    def fetch_data(self):
        """Fetch results"""
        pass

    def test_connection(self):
        try:
            self.connect()
            return "Connection successful!"
        except:
            return "Connection failed!"
        finally:
            self.disconnect()

class MySQLDatabase(Database):
    def __init__(self, host, user, password):
        self.host = host
        self.user = user
        self.password = password
        self.connection = None
        self.results = []

    def connect(self):

        print(f"Connecting to MySQL at {self.host}...")
        self.connection = "MySQL_Connection_Object"
        return True

    def disconnect(self):
```

```
print("Disconnecting from MySQL...")
self.connection = None

def execute_query(self, query):

    print(f"Executing MySQL query: {query}")
    self.results = [{"id": 1, "name": "John"}, {"id": 2, "name": "Jane"}]

def fetch_data(self):
    return self.results

class PostgreSQLDatabase(Database):
    def __init__(self, host, user, password):
        self.host = host
        self.user = user
        self.password = password
        self.connection = None
        self.results = []

    def connect(self):

        print(f"Connecting to PostgreSQL at {self.host}...")
        self.connection = "PostgreSQL_Connection_Object"
        return True

    def disconnect(self):
        print("Disconnecting from PostgreSQL...")
        self.connection = None

    def execute_query(self, query):

        print(f"Executing PostgreSQL query: {query}")
        self.results = [{"id": 1, "name": "John"}, {"id": 2, "name": "Jane"}]

    def fetch_data(self):
        return self.results

class MongoDBDatabase(Database):
    def __init__(self, host, user, password):
        self.host = host
        self.user = user
        self.password = password
        self.connection = None
        self.documents = []

    def connect(self):

        print(f"Connecting to MongoDB at {self.host}...")
        self.connection = "MongoDB_Connection_Object"
        return True
```

```
def disconnect(self):
    print("Disconnecting from MongoDB...")
    self.connection = None

def execute_query(self, query):

    print(f"Executing MongoDB query: {query}")
    self.documents = [{"_id": 1, "name": "John"}, {"_id": 2, "name": "Jane"}]

def fetch_data(self):
    return self.documents

def get_user_data(database):
    """Works with ANY database!"""
    database.connect()
    database.execute_query("SELECT * FROM users")
    data = database.fetch_data()
    database.disconnect()
    return data

mysql_db = MySQLDatabase("localhost", "root", "pass123")
postgre_db = PostgreSQLDatabase("localhost", "admin", "pass456")
mongo_db = MongoDBDatabase("localhost", "admin", "pass789")

print(get_user_data(mysql_db))
print(get_user_data(postgre_db))
print(get_user_data(mongo_db))
```

4. ABSTRACTION WITH PRIVATE MEMBERS

python

```
class BankAccount:
    def __init__(self, account_holder, initial_balance):
        self.account_holder = account_holder
        self.__balance = initial_balance
        self.__pin = "1234"
        self.__transaction_history = []

    def deposit(self, amount):
        if self.__validate_amount(amount):
            self.__balance += amount
            self.__log_transaction("deposit", amount)
            return f"Deposited ${amount}. New balance: ${self.get_balance()}"
        return "Invalid amount"

    def withdraw(self, amount, pin):
        if not self.__verify_pin(pin):
            return "Wrong PIN!"

        if self.__validate_amount(amount) and amount <= self.__balance:
            self.__balance -= amount
            self.__log_transaction("withdraw", amount)
            return f"Withdrew ${amount}. New balance: ${self.get_balance()}"
        return "Insufficient funds or invalid amount"

    def get_balance(self):
        return self.__balance

    def get_transaction_history(self, pin):
        if self.__verify_pin(pin):
            return self.__transaction_history.copy()
        return "Wrong PIN!"

    def __validate_amount(self, amount):
        return amount > 0 and isinstance(amount, (int, float))

    def __verify_pin(self, pin):
        return pin == self.__pin

    def __log_transaction(self, transaction_type, amount):
        from datetime import datetime
        self.__transaction_history.append({
            "type": transaction_type,
            "amount": amount,
            "time": datetime.now(),
            "balance": self.__balance
        })
```

```
account = BankAccount("Alice", 1000)

print(account.deposit(500))
print(account.withdraw(200, "1234"))
print(account.get_balance())

history = account.get_transaction_history("1234")
for entry in history:
    print(f"{entry['type']}: ${entry['amount']} at {entry['time']}")
```

5. ABSTRACTION LEVELS - From Simple to Complex

python

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass

    @abstractmethod
    def stop(self):
        pass

class Car(Vehicle):
    @abstractmethod
    def drive(self):
        pass

    def start(self):
        return "Car engine started"

    def stop(self):
        return "Car engine stopped"

class ElectricCar(Car):
    @abstractmethod
    def charge(self):
        pass

    def drive(self):
        return "Electric car driving silently"

    def start(self):
        return "Electric car powered on"

class Tesla(ElectricCar):
    def charge(self):
        return "Tesla charging at supercharger"

    def autopilot(self):
        return "Tesla autopilot engaged"

tesla = Tesla()
print(tesla.start())
print(tesla.drive())
print(tesla.charge())
```

```
print(tesla.autopilot())  
print(tesla.stop())
```

6. REAL INTERVIEW EXAMPLE: Payment Gateway Integration

python

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    def __init__(self, api_key):
        self.api_key = api_key
        self.__secret_key = "hidden_secret"

    @abstractmethod
    def process_payment(self, amount, currency):
        """Process payment - must be implemented"""
        pass

    @abstractmethod
    def refund_payment(self, transaction_id):
        """Refund payment - must be implemented"""
        pass

    @abstractmethod
    def verify_payment(self, transaction_id):
        """Verify payment - must be implemented"""
        pass

    def generate_receipt(self, transaction_id, amount):
        return {
            "transaction_id": transaction_id,
            "amount": amount,
            "timestamp": "2024-01-01",
            "gateway": self.__class__.__name__
        }

    def __encrypt_data(self, data):
        return f"encrypted_{data}"

    def _validate_amount(self, amount):
        return amount > 0

class StripeGateway(PaymentGateway):
    def process_payment(self, amount, currency):
        if not self._validate_amount(amount):
            return "Invalid amount"

        print(f"[Stripe] Processing {amount} {currency}")
        return {
            "status": "success",
```



```
        "transaction_id": f"STRIPE_{amount}",
        "amount": amount,
        "currency": currency
    }

    def refund_payment(self, transaction_id):
        print(f"[Stripe] Refunding {transaction_id}")
        return f"Stripe refund processed for {transaction_id}"

    def verify_payment(self, transaction_id):
        print(f"[Stripe] Verifying {transaction_id}")
        return f"Stripe payment {transaction_id} verified"

class PayPalGateway(PaymentGateway):
    def process_payment(self, amount, currency):
        if not self._validate_amount(amount):
            return "Invalid amount"

        print(f"[PayPal] Processing {amount} {currency}")
        return {
            "status": "success",
            "transaction_id": f"PAYPAL_{amount}",
            "amount": amount,
            "currency": currency
        }

    def refund_payment(self, transaction_id):
        print(f"[PayPal] Refunding {transaction_id}")
        return f"PayPal refund processed for {transaction_id}"

    def verify_payment(self, transaction_id):
        print(f"[PayPal] Verifying {transaction_id}")
        return f"PayPal payment {transaction_id} verified"

class RazorpayGateway(PaymentGateway):
    def process_payment(self, amount, currency):
        if not self._validate_amount(amount):
            return "Invalid amount"

        print(f"[Razorpay] Processing {amount} {currency}")
        return {
            "status": "success",
            "transaction_id": f"RZP_{amount}",
            "amount": amount,
            "currency": currency
        }

    def refund_payment(self, transaction_id):
        print(f"[Razorpay] Refunding {transaction_id}")
```

```
        return f"Razorpay refund processed for {transaction_id}"

    def verify_payment(self, transaction_id):
        print(f"[Razorpay] Verifying {transaction_id}")
        return f"Razorpay payment {transaction_id} verified"

class PaymentProcessor:
    def __init__(self, gateway: PaymentGateway):
        self.gateway = gateway

    def make_payment(self, amount, currency="USD"):

        result = self.gateway.process_payment(amount, currency)
        if result["status"] == "success":
            receipt = self.gateway.generate_receipt(
                result["transaction_id"],
                result["amount"]
            )
            return f"Payment successful! Receipt: {receipt}"
        return "Payment failed!"

    def refund(self, transaction_id):
        return self.gateway.refund_payment(transaction_id)

stripe = StripeGateway("stripe_api_key_123")
paypal = PayPalGateway("paypal_api_key_456")
razorpay = RazorpayGateway("razorpay_api_key_789")

processor1 = PaymentProcessor(stripe)
processor2 = PaymentProcessor(paypal)
processor3 = PaymentProcessor(razorpay)

print(processor1.make_payment(100))
print(processor2.make_payment(200))
print(processor3.make_payment(300))
```

7. DATA ABSTRACTION - Hiding Data Complexity

python

```
class Stack:
    def __init__(self):
        self.__items = []
        self.__size = 0

    def push(self, item):
        self.__items.append(item)
        self.__size += 1
        return f"Pushed: {item}"

    def pop(self):
        if not self.is_empty():
            self.__size -= 1
            return self.__items.pop()
        return "Stack is empty!"

    def peek(self):
        if not self.is_empty():
            return self.__items[-1]
        return "Stack is empty!"

    def is_empty(self):
        return self.__size == 0

    def size(self):
        return self.__size
```

```
stack = Stack()
stack.push(10)
stack.push(20)
stack.push(30)
print(stack.peek())
print(stack.pop())
print(stack.size())
```

```
class OptimizedStack:
    def __init__(self):
        self.__data = {}
        self.__top = -1

    def push(self, item):
        self.__top += 1
        self.__data[self.__top] = item
```

```
        return f"Pushed: {item}"

    def pop(self):
        if self.__top >= 0:
            item = self.__data[self.__top]
            del self.__data[self.__top]
            self.__top -= 1
            return item
        return "Stack is empty!"
```

8. COMMON INTERVIEW QUESTIONS & ANSWERS

Q1: What's the difference between Abstraction and Encapsulation?

python

```
class BankAccount:
    def __init__(self):
        self.__balance = 0

    def deposit(self, amount):
        self.__balance += amount

from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self):
        pass
```

Q2: Can we create object of Abstract Class?

python

```
from abc import ABC, abstractmethod

class AbstractClass(ABC):
    @abstractmethod
    def method(self):
        pass

class ConcreteClass(AbstractClass):
    def method(self):
        return "Implemented!"

obj = ConcreteClass()
```

Q3: What happens if child doesn't implement abstract method?

python

```
from abc import ABC, abstractmethod

class Parent(ABC):
    @abstractmethod
    def must_implement(self):
        pass

    @abstractmethod
    def also_must(self):
        pass

class Child(Parent):
    def must_implement(self):
        return "Done"

class GoodChild(Parent):
    def must_implement(self):
        return "Done"

    def also_must(self):
        return "Also done"

good_child = GoodChild()
```

Q4: Real-world example of Abstraction

python

```
from abc import ABC, abstractmethod

class EmailService(ABC):
    @abstractmethod
    def send_email(self, to, subject, body):
        pass

class GmailService(EmailService):
    def send_email(self, to, subject, body):

        print("Connecting to Gmail SMTP...")
        print("Authenticating...")
        print("Sending email...")
        return f"Email sent to {to} via Gmail"

class OutlookService(EmailService):
    def send_email(self, to, subject, body):

        print("Connecting to Outlook servers...")
        print("Using Microsoft Graph API...")
        print("Sending email...")
        return f"Email sent to {to} via Outlook"

def notify_user(email_service, user_email, message):
    return email_service.send_email(
        to=user_email,
        subject="Notification",
        body=message
    )

gmail = GmailService()
outlook = OutlookService()

print(notify_user(gmail, "user@gmail.com", "Hello!"))
print(notify_user(outlook, "user@outlook.com", "Hello!"))
```

9. DO's AND DON'Ts

 **DO:**

python

```
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class APIClient(ABC):
    @abstractmethod
    def fetch_data(self):
        pass

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

class Database(ABC):
    @abstractmethod
    def connect(self):
        pass

    def test(self):
        self.connect()
        print("Testing...")
        self.disconnect()
```

✗ DON'T:

python

```
class BadAbstract(ABC):
    @abstractmethod
    def method1(self):
        pass
    @abstractmethod
    def method2(self):
        pass
    @abstractmethod
    def method3(self):
        pass

class Bad(ABC):
    @abstractmethod
    def method(self):
        print("Implementation")

class Parent(ABC):
    @abstractmethod
    def method1(self):
        pass
    @abstractmethod
    def method2(self):
        pass

class Child(Parent):
    def method1(self):
        pass
```

10. QUICK CHEAT SHEET FOR INTERVIEW

11. VISUAL COMPARISON: ABSTRACTION IN REAL LIFE

python


```
class RestaurantMenu:
    def order_pizza(self, type):
        pass

    def pay_bill(self, amount):
        pass

class Kitchen:
    def __prepare_dough(self):
        pass

    def __add_toppings(self):
        pass

    def __bake_pizza(self):
        pass

    def __pack_pizza(self):
        pass
```

Remember This Mantra:

"Show WHAT it does, Hide HOW it does it!"

Key Interview Points:

1. **Abstraction = Hiding complexity, showing simplicity**
2. **Use ABC for enforcing structure**
3. **Private members for hiding data**
4. **Think "Remote Control" - Simple buttons, complex internals**
5. **Changes inside don't affect outside users**

Abstraction is like an iceberg:

- **What you see (10%):** Simple interface, public methods
- **What's hidden (90%):** Complex implementation, private methods, internal logic

The user only needs to see the tip, not the entire iceberg! 🏔️

Types of oops and explain very easy all concept

TYPES OF OOPs CONCEPTS - Complete Simple Guide

The 4 Pillars of OOP - Super Simple Definitions

Think of OOP like building with LEGO blocks!

1. ENCAPSULATION - "The Protective Box"

Simple Definition:

Wrapping data and methods together AND protecting them from outside interference

Real-Life Analogy:

- **Medicine Capsule:** Powder (data) is wrapped inside coating (protection)
- **ATM Machine:** Money inside, you can only access through card slot
- **School Bag:** Books inside, only you decide what to take out

Why Encapsulation?

- **Protection:** Nobody can directly mess with your data
- **Control:** You decide how data can be accessed/changed
- **Flexibility:** You can change inside without affecting outside

Code Examples:

python

```
class BankAccount_Bad:
```

```
    def __init__(self):  
        self.balance = 0
```

```
account = BankAccount_Bad()  
account.balance = -1000  
account.balance = "hack"  
print(account.balance)
```

```
class BankAccount:
```

```
    def __init__(self, account_holder):  
        self.account_holder = account_holder  
        self.__balance = 0  
        self.__pin = "1234"
```

```
    def deposit(self, amount):  
        if amount > 0:  
            self.__balance += amount  
            return f"✅ Deposited ${amount}. Balance: ${self.__balance}"  
        return "❌ Invalid amount!"
```

```
    def withdraw(self, amount, pin):  
        if pin != self.__pin:  
            return "❌ Wrong PIN!"  
        if amount > self.__balance:  
            return "❌ Insufficient funds!"  
        if amount <= 0:  
            return "❌ Invalid amount!"
```

```
        self.__balance -= amount  
        return f"✅ Withdrew ${amount}. Balance: ${self.__balance}"
```

```
    def get_balance(self, pin):  
        if pin == self.__pin:  
            return f"Balance: ${self.__balance}"  
        return "❌ Wrong PIN!"
```

```
account = BankAccount("Alice")  
print(account.account_holder)  
print(account.deposit(1000))  
print(account.withdraw(300, "1234"))
```

```
print(account.get_balance("1234"))  
print(account.get_balance("0000"))
```

Access Modifiers in Python:

python

```
class AccessModifiersDemo:  
    def __init__(self):  
        self.public_var = "I'm PUBLIC - Anyone can access me!"  
        self._protected_var = "I'm PROTECTED - Please don't touch!"  
        self.__private_var = "I'm PRIVATE - Keep out!"  
  
    def public_method(self):  
        return "Public method - Use me freely!"  
  
    def _protected_method(self):  
        return "Protected method - For internal use only"  
  
    def __private_method(self):  
        return "Private method - Nobody outside can call me"  
  
    def access_private(self):  
  
        return self.__private_var  
  
obj = AccessModifiersDemo()  
  
print(obj.public_var)  
print(obj.public_method())  
  
print(obj._protected_var)  
print(obj._protected_method())  
  
print(obj._AccessModifiersDemo__private_var)
```

2. INHERITANCE - "Like Parent, Like Child"

Simple Definition:

Child class inherits (gets) all properties and methods from parent class

Real-Life Analogy:

- **Family:** You inherit your father's height and mother's eyes
- **Vehicle:** Car, Bike, Truck all are vehicles (inherit basic features)
- **Animals:** Dog, Cat, Bird all are animals (share common features)

Why Inheritance?

- **Code Reuse:** Write once, use multiple times
- **Hierarchy:** Shows relationship between classes
- **Extension:** Add new features to existing code

Types of Inheritance:

python

```
print("=" * 50)
print("1. SINGLE INHERITANCE")
print("=" * 50)

class Animal:
    def eat(self):
        return "Eating..."

    def sleep(self):
        return "Sleeping..."

class Dog(Animal):
    def bark(self):
        return "Woof!"

dog = Dog()
print(dog.eat())
print(dog.sleep())
print(dog.bark())

print("\n" + "=" * 50)
print("2. MULTILEVEL INHERITANCE")
print("=" * 50)

class Grandfather:
    def property(self):
        return "Land and House"

class Father(Grandfather):
    def business(self):
        return "Shop"

class Son(Father):
    def education(self):
        return "Engineering"

son = Son()
print(son.property())
print(son.business())
print(son.education())

print("\n" + "=" * 50)
print("3. HIERARCHICAL INHERITANCE")
print("=" * 50)
```

```
class Vehicle:
    def move(self):
        return "Moving..."

class Car(Vehicle):
    def drive(self):
        return "Driving on road"

class Boat(Vehicle):
    def sail(self):
        return "Sailing on water"

class Plane(Vehicle):
    def fly(self):
        return "Flying in air"

car = Car()
boat = Boat()
plane = Plane()

print(car.move())
print(boat.move())
print(plane.move())

print("\n" + "=" * 50)
print("4. MULTIPLE INHERITANCE")
print("=" * 50)

class Father:
    def height(self):
        return "Tall (6 feet)"

    def skill(self):
        return "Cooking"

class Mother:
    def intelligence(self):
        return "Very Smart"

    def skill(self):
        return "Painting"

class Child(Father, Mother):
    def creativity(self):
        return "Creative"

child = Child()
print(child.height())
print(child.intelligence())
```

```
print(child.skill())
print(child.creativity())

print(Child.__mro__)

print("\n" + "=" * 50)
print("5. HYBRID INHERITANCE")
print("=" * 50)

class ElectronicDevice:
    def power(self):
        return "Can be turned on/off"

class Phone(ElectronicDevice):
    def call(self):
        return "Can make calls"

class SmartDevice(ElectronicDevice):
    def connect_wifi(self):
        return "Can connect to WiFi"

class SmartPhone(Phone, SmartDevice):
    def browse_internet(self):
        return "Can browse internet"

smartphone = SmartPhone()
print(smartphone.power())
print(smartphone.call())
print(smartphone.connect_wifi())
print(smartphone.browse_internet())
```

super() - Call Your Parent!

python


```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print(f"Person created: {name}, {age} years old")

    def introduce(self):
        return f"Hi, I'm {self.name}"

class Student(Person):
    def __init__(self, name, age, student_id, grade):

        super().__init__(name, age)
        self.student_id = student_id
        self.grade = grade
        print(f"Student ID: {student_id}, Grade: {grade}")

    def introduce(self):

        parent_intro = super().introduce()
        return f"{parent_intro}, Student ID: {self.student_id}"

class Teacher(Person):
    def __init__(self, name, age, subject, salary):
        super().__init__(name, age)
        self.subject = subject
        self.salary = salary

    def introduce(self):
        return f"I'm {self.name}, I teach {self.subject}"

student = Student("Alice", 20, "ST123", "A")
teacher = Teacher("Mr. Smith", 35, "Math", 50000)

print(student.introduce())
print(teacher.introduce())
```

3. POLYMORPHISM - "One Name, Many Forms"

Simple Definition:

| Same method name behaves differently for different classes

Real-Life Analogy:

- "Play" Button:
 - On TV → Plays video

- On Music Player → Plays song
- On Game Console → Plays game
- **You as a person:**
 - At home → Son/Daughter
 - At work → Employee
 - At party → Friend
- **"Open" action:**
 - Open a book → Read pages
 - Open a door → Enter room
 - Open a file → See contents

Why Polymorphism?

- **Flexibility:** Same interface, different behaviors
- **Simplicity:** One method name to remember
- **Extensibility:** Add new classes without changing code

Types of Polymorphism:

python

```
print("=" * 50)
print("1. METHOD OVERRIDING")
print("=" * 50)

class Shape:
    def area(self):
        return 0

    def perimeter(self):
        return 0

    def describe(self):
        return f"Shape - Area: {self.area()}, Perimeter: {self.perimeter()}"

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

    def perimeter(self):
        return 2 * 3.14 * self.radius

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

class Triangle(Shape):
    def __init__(self, base, height, side1, side2):
        self.base = base
        self.height = height
        self.side1 = side1
        self.side2 = side2

    def area(self):
        return 0.5 * self.base * self.height

    def perimeter(self):
        return self.base + self.side1 + self.side2
```

```
shapes = [Circle(5), Rectangle(4, 6), Triangle(3, 4, 5, 5)]
```

```
for shape in shapes:  
    print(shape.describe())
```

```
print("\n")
```

```
print("=" * 50)  
print("2. DUCK TYPING")  
print("=" * 50)
```

```
class Duck:  
    def speak(self):  
        return "Quack Quack!"  
  
    def fly(self):  
        return "Duck flying"
```

```
class Person:  
    def speak(self):  
        return "Hello! I can quack too!"  
  
    def fly(self):  
        return "Person in airplane"  
  
    def walk(self):  
        return "Walking"
```

```
class Dog:  
    def speak(self):  
        return "Woof Woof!"  
  
    def fly(self):  
        return "Dogs can't fly!"  
  
    def wag_tail(self):  
        return "Wagging tail"
```

```
def make_it_speak_and_fly(thing):  
    print(f"Speaking: {thing.speak()}")  
    print(f"Flying: {thing.fly()}")  
    print("---")
```

```
make_it_speak_and_fly(Duck())  
make_it_speak_and_fly(Person())  
make_it_speak_and_fly(Dog())
```

```
print("\n")

print("=" * 50)
print("3. OPERATOR OVERLOADING")
print("=" * 50)

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Point(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar):
        return Point(self.x * scalar, self.y * scalar)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __str__(self):
        return f"Point({self.x}, {self.y})"

    def __len__(self):
        return 2

p1 = Point(2, 3)
p2 = Point(4, 5)

print(p1 + p2)
print(p1 - p2)
print(p1 * 3)
print(p1 == p2)
print(p1 == Point(2, 3))
print(len(p1))

print("\n")

print("=" * 50)
print("4. METHOD OVERLOADING (Simulated)")
print("=" * 50)

class Calculator:

    def add(self, a, b, c=0, d=0):
```

```
        return a + b + c + d

    def multiply(self, *args):
        result = 1
        for num in args:
            result *= num
        return result

calc = Calculator()

print(calc.add(10, 20))
print(calc.add(10, 20, 30))
print(calc.add(10, 20, 30, 40))

print(calc.multiply(2, 3))
print(calc.multiply(2, 3, 4))
print(calc.multiply(2, 3, 4, 5))
```

4. ABSTRACTION - "Hide Complexity, Show Simplicity"

Simple Definition:

Show only what's necessary, hide complex implementation details

Real-Life Analogy:

- **Car Dashboard:** You see speed, fuel, but not engine internals
- **Coffee Machine:** You press a button, don't need to know internal process
- **Smartphone:** You tap screen, don't need to know circuit board logic
- **Restaurant Menu:** You see dishes, not recipes or kitchen process

Why Abstraction?

- **Simplicity:** Users don't need complex knowledge
- **Security:** Hide sensitive implementation
- **Maintenance:** Change internals without affecting users
- **Focus:** Concentrate on WHAT, not HOW

Implementing Abstraction:

python

```
from abc import ABC, abstractmethod

print("=" * 50)
print("1. ABSTRACT CLASS")
print("=" * 50)

class Vehicle(ABC):
    def __init__(self, brand):
        self.brand = brand

    @abstractmethod
    def start(self):
        """Every vehicle must have start method"""
        pass

    @abstractmethod
    def stop(self):
        """Every vehicle must have stop method"""
        pass

    def honk(self):
        return "Beep Beep!"

    def info(self):
        return f"Brand: {self.brand}"

class Car(Vehicle):
    def start(self):
        return f"{self.brand} car: Insert key and turn"

    def stop(self):
        return f"{self.brand} car: Press brake and stop"

class Bike(Vehicle):
    def start(self):
        return f"{self.brand} bike: Kick start"

    def stop(self):
        return f"{self.brand} bike: Apply brake"

class ElectricCar(Vehicle):
    def start(self):
        return f"{self.brand} electric car: Press button"
```

```

def stop(self):
    return f"{self.brand} electric car: Auto stop"

vehicles = [Car("Toyota"), Bike("Honda"), ElectricCar("Tesla")]

for v in vehicles:
    print(v.start())
    print(v.stop())
    print(v.honk())
    print("---")

print("\n")

print("=" * 50)
print("2. PAYMENT SYSTEM ABSTRACTION")
print("=" * 50)

class PaymentProcessor(ABC):
    @abstractmethod
    def process_payment(self, amount):
        pass

    @abstractmethod
    def refund(self, transaction_id):
        pass

    def generate_receipt(self, amount, transaction_id):
        return {
            "amount": amount,
            "transaction_id": transaction_id,
            "status": "completed"
        }

class CreditCardPayment(PaymentProcessor):
    def __init__(self, card_number):
        self.__card_number = card_number

    def process_payment(self, amount):

        print(f"[Credit Card] Processing ${amount} from card ****{self.__card_number[-4:]}")
        return f"CC_{amount}"

    def refund(self, transaction_id):
        print(f"[Credit Card] Refunding {transaction_id}")
        return f"Refund: {transaction_id}"

class PayPalPayment(PaymentProcessor):
    def __init__(self, email):

```



```

        self.__email = email

    def process_payment(self, amount):

        print(f"[PayPal] Processing ${amount} from {self.__email}")
        return f"PP_{amount}"

    def refund(self, transaction_id):
        print(f"[PayPal] Refunding {transaction_id}")
        return f"Refund: {transaction_id}"

class CryptoPayment(PaymentProcessor):
    def __init__(self, wallet):
        self.__wallet = wallet

    def process_payment(self, amount):

        print(f"[Crypto] Processing ${amount} from wallet {self.__wallet[:10]}...")
        return f"CRYPTO_{amount}"

    def refund(self, transaction_id):
        print(f"[Crypto] Refunding {transaction_id}")
        return f"Refund: {transaction_id}"

def checkout(payment_processor, amount):
    print("Starting checkout...")
    transaction_id = payment_processor.process_payment(amount)
    receipt = payment_processor.generate_receipt(amount, transaction_id)
    print(f"✅ Payment successful! Receipt: {receipt}")
    return transaction_id

credit = CreditCardPayment("1234567890123456")
paypal = PayPalPayment("user@email.com")
crypto = CryptoPayment("0x1234567890abcdef")

checkout(credit, 100)
print()
checkout(paypal, 200)
print()
checkout(crypto, 300)

```

5. COMPLETE REAL-WORLD EXAMPLE - All 4 Pillars Together

python

```
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, name, id, base_salary):

        self.name = name
        self.id = id
        self.__base_salary = base_salary
        self.__bonus = 0

    @abstractmethod
    def calculate_salary(self):
        """Each employee type calculates differently"""
        pass

    @abstractmethod
    def get_role(self):
        """Each employee has different role"""
        pass

    def set_bonus(self, amount):
        if amount > 0:
            self.__bonus = amount

    def get_base_salary(self):
        return self.__base_salary

    def get_bonus(self):
        return self.__bonus

    def get_details(self):
        return f"{self.name} ({self.id})"

class Manager(Employee):
    def __init__(self, name, id, base_salary, team_size):
        super().__init__(name, id, base_salary)
        self.team_size = team_size

    def calculate_salary(self):
        team_bonus = self.team_size * 500
        return self.get_base_salary() + self.get_bonus() + team_bonus

    def get_role(self):
        return "Manager"
```

```
class Developer(Employee):
    def __init__(self, name, id, base_salary, skills):
        super().__init__(name, id, base_salary)
        self.skills = skills

    def calculate_salary(self):
        skill_bonus = len(self.skills) * 1000
        return self.get_base_salary() + self.get_bonus() + skill_bonus

    def get_role(self):
        return "Developer"

class Intern(Employee):

    def calculate_salary(self):
        return self.get_base_salary() + self.get_bonus()

    def get_role(self):
        return "Intern"

print("=" * 60)
print("COMPANY EMPLOYEE MANAGEMENT SYSTEM")
print("=" * 60)

employees = [
    Manager("Alice", "M001", 80000, 5),
    Developer("Bob", "D001", 65000, ["Python", "Java", "React"]),
    Developer("Charlie", "D002", 70000, ["Python", "Django", "AWS", "Docker"]),
    Intern("David", "I001", 20000)
]

employees[0].set_bonus(5000)
employees[1].set_bonus(3000)
employees[2].set_bonus(4000)

total_salary = 0
for emp in employees:
    salary = emp.calculate_salary()
    total_salary += salary
    print(f"Role: {emp.get_role():10} | {emp.get_details():20} | Salary: ${salary:>8}")

print(f"\nTotal Monthly Salary Expense: ${total_salary}")
```

Output:

text

```
=====
COMPANY EMPLOYEE MANAGEMENT SYSTEM
=====
Role: Manager   | Alice (M001)      | Salary: $ 87500
Role: Developer | Bob (D001)         | Salary: $ 71000
Role: Developer | Charlie (D002)     | Salary: $ 78000
Role: Intern    | David (I001)       | Salary: $ 20000

Total Monthly Salary Expense: $256500
```

6. QUICK COMPARISON - All 4 Pillars at a Glance

python

```
from abc import ABC, abstractmethod

class Smartphone(ABC):
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
        self.__battery = 100
        self.__storage = 128

    @abstractmethod
    def take_photo(self):
        pass

    def charge(self, amount):
        self.__battery = min(100, self.__battery + amount)

    def get_battery(self):
        return self.__battery

class AndroidPhone(Smartphone):
    def take_photo(self):
        return "📷 Android photo clicked with AI enhancement"

class iPhone(Smartphone):
    def take_photo(self):
        return "📷 iPhone photo clicked with Deep Fusion"

def test_camera(phone):
    return phone.take_photo()

android = AndroidPhone("Samsung", "S24")
iphone = iPhone("Apple", "15 Pro")

print(test_camera(android))
print(test_camera(iphone))
```

7. COMMON INTERVIEW QUESTIONS

Q1: What are the 4 pillars of OOP?

- **Encapsulation:** Wrapping data and methods, hiding internal state
- **Inheritance:** Child class inherits from parent class
- **Polymorphism:** Same interface, different implementations

- **Abstraction:** Hiding complexity, showing only essentials

Q2: Difference between Abstraction and Encapsulation?

python

```
class Example:
    def __init__(self):
        self.__data = 0

    def get_data(self):
        return self.__data

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Q3: What is the difference between Abstract Class and Interface?

python

```
from abc import ABC, abstractmethod

class AbstractClass(ABC):
    @abstractmethod
    def method1(self):
        pass

    def method2(self):
        return "Concrete method"

class Interface(ABC):
    @abstractmethod
    def method1(self):
        pass

    @abstractmethod
    def method2(self):
        pass
```

8. REMEMBER THIS MANTRA

9. DO's AND DON'Ts SUMMARY

✓ DO:

- Use encapsulation to protect data (private variables)
- Use inheritance only when "IS-A" relationship exists
- Use polymorphism for interchangeable components
- Use abstraction to define clear interfaces
- Keep class hierarchies shallow (max 3 levels)

✗ DON'T:

- Don't expose private data directly
- Don't use inheritance for "HAS-A" relationship (use composition)
- Don't create deep inheritance chains
- Don't check types manually (use polymorphism instead)
- Don't put implementation in abstract methods

Remember: OOP is like building with LEGO blocks - each piece (class) has its own purpose, and together they create amazing structures! 🏰